

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего
образования

«Московский политехнический университет»

Образовательная программа «Веб-технологии»

Направление подготовки: 09.03.01 «Информатика и вычислительная техника»

Отчёт по курсовой работе

по дисциплине «Серверная веб-разработка»

Тема: «Backend-фреймворк на PHP. Сайт приюта для животных.»

Выполнил:

Студентка группы 257-321
Кашафутдинова А.Т.

подпись, дата

Принял:

Преподаватель
Баринов В.Р.

подпись, дата

Москва, 2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
Глава 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	5
1.1 Анализ предметной области	5
1.2 Анализ аналогов.....	5
1.3 Используемые технологии.....	7
1.4 Архитектурные паттерны	8
1.5 Анализ функциональности	8
Глава 2. ПРАКТИЧЕСКАЯ ЧАСТЬ.....	10
2.1 Проектирование базы данных	10
2.2 Архитектура приложения и маршрутизация	11
2.3 Административная часть	13
2.4 Главная и детальные страницы	16
2.5 Дополнительный функционал	19
2.6 Безопасность	21
ЗАКЛЮЧЕНИЕ.....	23
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.....	25
ПРИЛОЖЕНИЕ А.....	26
А.1 Таблица users — пользователи	26
А.2 Таблица pets — питомцы	27
А.3 Таблица articles — статьи блога	27
А.4 Таблица feedback — обращения.....	28
А.5 SQL-листинг создания таблиц.....	28
ПРИЛОЖЕНИЕ Б	31

ВВЕДЕНИЕ

Актуальность темы. Проблема бездомных животных остро стоит в крупных городах, и значительную роль в её решении играют приюты. По различным оценкам, в России насчитывается несколько сотен зоозащитных организаций, через которые ежегодно проходят десятки тысяч животных. Эффективность работы приюта во многом определяется его информационной средой: насколько удобно потенциальные владельцы могут познакомиться с питомцами, какие сервисы им предоставляются, и насколько просто сотрудники могут вести учёт своих подопечных. Современный сайт приюта — это не просто «визитка», а полноценный инструмент, объединяющий витрину каталога, форму взаимодействия с посетителями, сервис расчётов и панель администрирования.

Объект исследования: процессы информационного взаимодействия посетителя сайта приюта и сотрудника, ведущего учёт животных и контента.

Предмет исследования: программная реализация веб-сайта приюта на собственном backend-фреймворке, объединяющего публичную часть и административную панель управления.

Цель проекта: разработать веб-сайт приюта для животных на основе самостоятельно реализованного backend-фреймворка на языке PHP, который объединяет в единый продукт все технологии, изученные в курсе «Серверная веб-разработка»: динамические формы, работу с реляционной базой данных, операции CRUD, маршрутизацию, шаблонизацию и объектно-реляционное отображение.

Задачи проекта:

1. проанализировать предметную область и пользовательские сценарии;
2. проанализировать аналогичные веб-сайты и выявить лучшие практики;
3. выбрать и обосновать применяемые технологии и архитектурные паттерны;

4. спроектировать структуру реляционной базы данных и связи между сущностями;
5. реализовать ядро backend-фреймворка: фронт-контроллер, маршрутизацию, базовый класс ActiveRecord, компонент представления, сервис аутентификации;
6. разработать административную часть с авторизацией и полным набором CRUD-операций для управления питомцами и статьями;
7. разработать публичную часть: главную, каталог, детальные страницы, статьи, форму обратной связи, страницу служебной информации;
8. реализовать дополнительный функционал: интерактивный калькулятор нормы корма;
9. обеспечить базовую безопасность (хеширование паролей, подготовленные выражения, разграничение доступа);
10. развернуть проект, оформить документацию и загрузить исходный код в удалённый репозиторий.

Новизна работы. Backend-фреймворк реализован самостоятельно, без использования сторонних библиотек уровня Laravel или Symfony, что демонстрирует понимание базовых принципов архитектуры современных веб-приложений и обеспечивает компактность и обзримость кода.

Практическая значимость. Архитектура проекта легко расширяется: добавление новых сущностей сводится к созданию класса-наследника ActiveRecordEntity и регистрации маршрутов. Сайт может быть адаптирован для реального приюта при сравнительно небольших доработках (загрузка фотографий, многоязычность, интеграция с платёжной системой для пожертвований).

Ссылка на репозиторий:

https://github.com/alinakashafutdinova/Lapki_kursovaya.git.

Глава 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Анализ предметной области

Предметная область проекта — деятельность приюта для бездомных животных и информационное сопровождение процесса пристройства питомцев в семьи. Участники процесса делятся на три категории: потенциальные владельцы, волонтеры и сотрудники приюта. У каждой категории — собственный набор сценариев работы с сайтом.

Потенциальные владельцы хотят быстро увидеть фотографии и характеристики доступных животных, отфильтровать каталог по виду, прочитать историю заинтересовавшего питомца и оставить заявку. Дополнительно им полезны сервисы, помогающие оценить ответственность будущего владения — например, расчёт необходимого количества корма.

Волонтеры ищут информацию о том, чем можно помочь приюту: выгул, помощь по хозяйству, перевозки. Им важна форма обратной связи с возможностью указать тип обращения.

Сотрудники приюта нуждаются в инструменте для ведения базы животных (добавление, редактирование, изменение статуса) и публикации статей. Им нужно простое разграничение доступа: гость видит только публичную часть, сотрудник с правами администратора получает доступ к панели управления.

Из перечисленных сценариев формируется набор обязательных функциональных модулей: каталог питомцев, детальная карточка, раздел статей, форма обратной связи, административная панель с CRUD-операциями. Дополнительные модули — калькулятор и служебная страница с HTTP-заголовками — выделяют сайт среди типовых ресурсов и демонстрируют расширенный спектр изученных технологий. Таким образом, эти модули закроют потребности всех участников и повысят эффективность пристройства. Сайт превратится из простой витрины в удобный инструмент для приюта и будущих хозяев.

1.2 Анализ аналогов

Так как проект является некоммерческим (приют), корректнее говорить не о конкурентах, а об аналогах — других веб-сайтах приютов и зоозащитных платформ. Для анализа выбраны три типичных представителя данной категории.

11. Зоозащитный портал крупного российского фонда — отличается обширным каталогом и системой пожертвований, но при этом перегружен баннерами, а карточки питомцев имеют разный уровень оформления.

12. Региональный сайт небольшого приюта — простая статичная вёрстка, телефон администратора как основной способ связи; отсутствует онлайн-форма и фильтрация.

13. Агрегатор объявлений о пристроении — современный дизайн и удобный поиск, но не предоставляет редакторских материалов о уходе и не имеет инструмента-калькулятора.

Из проведённого анализа сделаны выводы. Сильные стороны, которые следует перенять: наглядные карточки с фотографиями, фильтрация по виду, форма обратной связи. Слабые стороны, которых следует избегать: визуальная перегруженность, отсутствие адаптивной вёрстки, разобщённость функциональных блоков. Сравнение основных характеристик приведено в таблице 1.

Таблица 1. Сравнение проекта «Лапки» с аналогами

Характеристика	Зоо-портал	Сайт приюта	Агрегатор	«Лапки»
Каталог с фильтрами	+	–	+	+
Детальные карточки	+	+/-	+	+
Адаптивный дизайн	+/-	–	+	+
Форма обратной связи	+	–	+	+
Сервис-калькулятор	–	–	–	+
Раздел статей	+	–	+/-	+
Панель администрирования	+	–	+	+

Видно, что проект «Лапки» вобрал в себя ключевые сильные стороны аналогов и дополнил их уникальной для категории функцией — калькулятором

нормы корма, помогающим будущему владельцу оценить заботу о питомце ещё до знакомства.

1.3 Используемые технологии

Язык PHP. В качестве серверного языка использован PHP версии 8, поддерживаемый практически любым хостингом. PHP сочетает простоту синтаксиса с обширной стандартной библиотекой и широкой экосистемой. В проекте задействованы: типизация параметров и возвращаемых значений, пространства имён, классы и интерфейсы, рефлексия (для динамического сохранения свойств в ActiveRecord), сессии (для хранения авторизации), хеш-функции `password_hash` и `password_verify`, функция `get_headers` для демонстрации работы с HTTP.

СУБД MySQL. Хранение данных организовано в реляционной СУБД MySQL (совместимой реализации MariaDB). Используется кодировка `utf8` и движок таблиц `InnoDB`, поддерживающий внешние ключи. Все запросы выполняются через расширение `PDO` с применением подготовленных выражений (`prepared statements`) — это защищает приложение от SQL-инъекций.

Веб-сервер Apache. Для развёртывания используется Apache с включённым модулем `mod_rewrite`. Файл `.htaccess` в корне проекта реализует механизм «человекопонятных адресов» (ЧПУ): неизвестные пути перенаправляются на фронт-контроллер с передачей запрошенного маршрута в параметре `route`.

HTML и CSS. Клиентская часть сформирована средствами HTML5 и каскадных таблиц стилей. В оформлении применяются CSS-переменные для единой палитры, флекс- и грид-раскладки для адаптивности, медиа-запросы. Шрифтовые пары `Baloo 2` и `Onest` подключаются с сервиса `Google Fonts`.

Контроль версий и развёртывание. Исходный код проекта размещается в системе контроля версий `Git` и публикуется в удалённом репозитории `GitHub`. На

целевой хостинг проект разворачивается командой `git clone` с последующей настройкой подключения к базе данных в файле `src/settings.php`.

1.4 Архитектурные паттерны

В основе серверной части лежат три классических паттерна.

Model–View–Controller (MVC). Код разделён на три слоя: модели описывают сущности предметной области и их сохранение; представления (View) формируют HTML-страницы; контроллеры обрабатывают запрос, обращаются к моделям и передают данные в представление. Такое разделение упрощает поддержку и расширение проекта: изменение оформления страниц затрагивает только шаблоны, новая бизнес-логика — только контроллеры.

Front Controller (фронт-контроллер). Все запросы к сайту приходят в единственный файл `index.php`, который определяет, какой контроллер и какое действие должны быть вызваны. Это обеспечивает единое место для аутентификации, обработки ошибок и подключения автозагрузчика классов.

Active Record. Каждой таблице БД соответствует класс-сущность (User, Pet, Article, Feedback). Объект такого класса инкапсулирует одну запись таблицы и предоставляет методы `save()`, `delete()` и статические `findAll()`, `getById()`, `findOneByColumn()`. Базовый класс `ActiveRecordEntity` реализует общую логику через рефлексия, поэтому конкретным сущностям достаточно объявить свойства и метод `getTableName()`.

1.5 Анализ функциональности

Функциональные возможности сайта разделены на публичную часть и административную панель. Публичная часть доступна любому посетителю, административная — только авторизованному пользователю с ролью администратора.

Публичная часть включает следующие функции:

- главная страница: приветственный блок со статистикой приюта, подборка питомцев и последних статей;
- каталог питомцев: карточки с фотографиями, фильтрация по виду животного;
- детальная карточка питомца: характеристики, описание, переход к калькулятору с заранее заполненными параметрами;
- раздел статей: список и детальная страница с динамическим заголовком вкладки браузера и указанием автора;
- калькулятор суточной нормы корма: расчёт по виду, весу, возрасту и активности;
- форма обратной связи: имя, e-mail, тип обращения, текст, выбор способа ответа; данные сохраняются в БД;
- страница HTTP-заголовков для произвольного URL (демонстрация функции `get_headers`);
- обработка несуществующих адресов: страница 404 с сохранением общей вёрстки сайта.

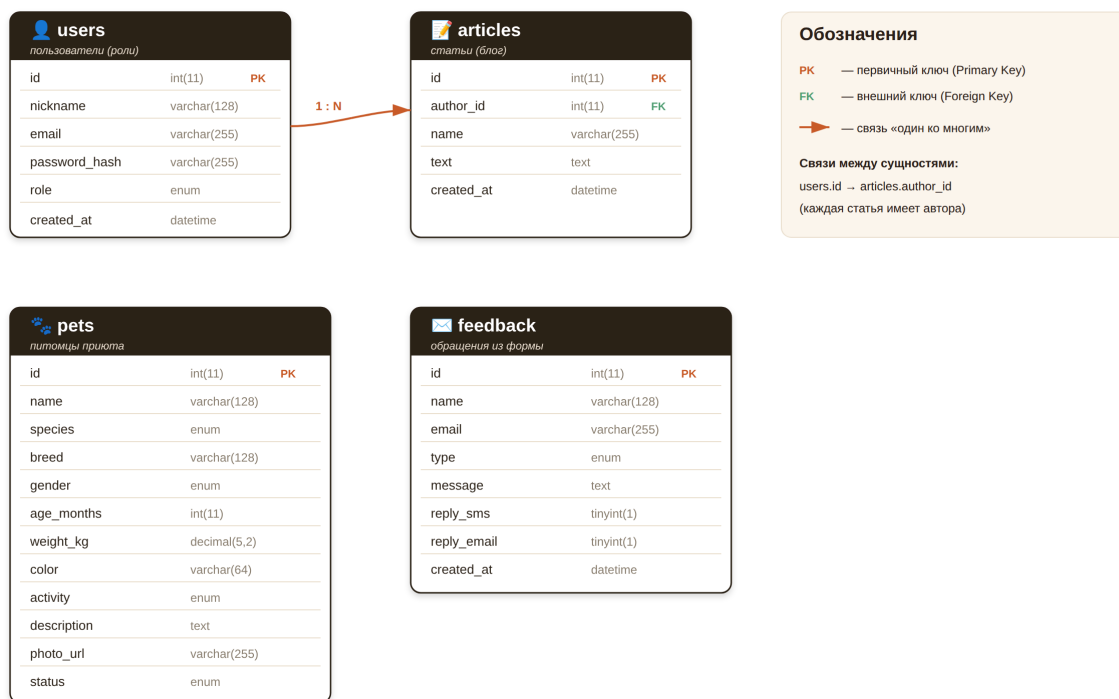
Административная панель включает:

- вход по e-mail и паролю с проверкой хеша `password_verify`, создание сессии;
- дашборд со сводной статистикой: число питомцев, количество ищущих дом, число обращений;
- управление питомцами: таблица с пагинацией по 10 записей и сортировкой по дате, кличке, возрасту и статусу, формы добавления и редактирования (12 атрибутов), удаление с подтверждением;
- управление статьями: список, формы добавления и редактирования, удаление с подтверждением; автором статьи автоматически назначается текущий администратор;
- просмотр обращений из формы обратной связи в едином списке.

Глава 2. ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Проектирование базы данных

База данных проекта реализована в СУБД MySQL и состоит из четырёх таблиц. Структура базы данных в виде ER-диаграммы приведена на рисунке 1.



ER-диаграмма базы данных приюта «Лапки»

Рисунок 1. ER-диаграмма базы данных приюта «Лапки»

Каждая таблица решает свою задачу. Сводное описание представлено в таблице 2; подробное описание полей каждой таблицы вынесено в Приложение А.

Таблица 2. Назначение таблиц базы данных

Таблица	Назначение	Ключевые поля
users	Пользователи системы и их роли (admin/user). Используется для авторизации и определения автора статьи.	id (PK), nickname, email, password_hash, role
pets	Карточки питомцев приюта: атрибуты животного и его статус.	id (PK), name, species, gender, age_months, weight_kg, status
articles	Статьи блога. Связаны с автором через author_id → users.id.	id (PK), author_id (FK), name, text
feedback	Обращения, поступившие через форму обратной связи.	id (PK), name, email, type, message, reply_sms, reply_email

Между таблицами установлена одна явная связь: внешний ключ `articles.author_id` ссылается на `users.id` (отношение «один ко многим»: один пользователь может быть автором нескольких статей). На уровне приложения связь реализуется дополнительным запросом из модели `User` по идентификатору автора, что демонстрирует характерный для Active Record подход к выборке связанных данных.

Таблицы `pets` и `feedback` не имеют внешних ключей, что отражает реальную предметную область: учётная запись питомца не требует обязательной связи с пользователем (питомцев заводит администратор, но не «владеет» ими), а обращения посетителей оставляют гости сайта, не имеющие учётной записи.

Все запросы выполняются через подготовленные выражения PDO, что исключает возможность SQL-инъекций. Кодировка таблиц — `utf8` (`utf8_general_ci`), движок — InnoDB. Полный SQL-листинг создания таблиц приведён в Приложении А.

2.2 Архитектура приложения и маршрутизация

Приложение построено по архитектурному паттерну Model–View–Controller с использованием паттерна Front Controller. Все запросы к сайту приходят в единый файл `index.php`, расположенный в корне проекта. Скрипт сразу запускает сессию (для надёжного хранения авторизации между запросами), затем регистрирует автозагрузчик классов и подключает таблицу маршрутов.

```
<?php

// Сессию стартуем в самом начале, до любого вывода —
// это гарантирует надёжное сохранение авторизации между запросами.
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

try {
    spl_autoload_register(function (string $className) {
        $className = str_replace('\\', '/', $className);
        require_once __DIR__ . '/src/' . $className . '.php';
    });

    $route = $_GET['route'] ?? '';
    $routes = require __DIR__ . '/src/routes.php';

    $isRouteFound = false;
    foreach ($routes as $pattern => $controllerAndAction) {
        preg_match($pattern, $route, $matches);
    }
}
```

```

        if (!empty($matches)) {
            $isRouteFound = true;
            break;
        }
    }

```

Листинг 1. Запуск сессии, автозагрузчик и обработка маршрута во фронт-контроллере

Таблица маршрутов `src/routes.php` — это ассоциативный массив, в котором ключом служит регулярное выражение, а значением — пара «контроллер — действие». Захваченные группы регулярного выражения передаются в действие в качестве аргументов (через распаковку массива в PHP). Такой подход позволяет строить «человекопонятные» URL вида `/pets/5` или `/articles/2`, а также параметризованные демо-руты `/hello/Имя` и `/bye/Имя`.

```

'^^pets$~'           => [PetsController::class, 'list'],
'^^pets/(\d+)$~'     => [PetsController::class, 'show'],
'^^articles$~'       => [ArticlesController::class, 'list'],
'^^articles/(\d+)$~' => [ArticlesController::class, 'show'],
'^^calculator$~'     => [CalculatorController::class, 'feed'],
'^^feedback$~'       => [FeedbackController::class, 'form'],
'^^feedback/submit$~' => [FeedbackController::class, 'submit'],
'^^hello/(.+)$~'     => [MainController::class, 'sayHello'],
'^^bye/(.+)$~'       => [MainController::class, 'sayBye'],
'^^admin/pets$~'     => [AdminController::class, 'petsList'],
'^^admin/articles$~' => [AdminController::class, 'articlesList'],

```

Листинг 2. Фрагмент таблицы маршрутов `src/routes.php`

Базовый класс `ActiveRecordEntity` инкапсулирует типовые операции с таблицами БД. Он реализует магический сеттер для автоматического преобразования имён столбцов (`snake_case`) в имена свойств класса (`camelCase`), методы сохранения `save()` (с автоматическим выбором INSERT или UPDATE), удаления `delete()`, а также статические методы выборки `findAll()`, `getById()`, `findOneByColumn()`. Конкретным сущностям остаётся лишь объявить свойства и реализовать метод `getTableName()`.

```

public function save(): void {
    $mapped = $this->mapPropertiesToDbFormat();
    if ($this->id !== null) { $this->update($mapped); }
    else { $this->insert($mapped); }
}

public static function getById(int $id): ?self {
    $db = new Db();
    $entities = $db->query(
        'SELECT * FROM `'. static::getTableName() . '` WHERE id = :id;',
        [':id' => $id], static::class
    );
    return $entities ? $entities[0] : null;
}

```

Листинг 3. Ключевые методы базового класса `ActiveRecordEntity`

Компонент представления View реализует метод `renderHtml($templateName, $vars, $code)`. Он распаковывает переменные в локальный контекст с помощью функции `extract`, подключает шаблон в буфер вывода и отдаёт результат клиенту. Это позволяет шаблонам пользоваться переменными напрямую без явного обращения к массиву данных.

2.3 Административная часть

Административная панель защищена авторизацией. Форма входа (рисунок 2) принимает e-mail и пароль; пароль сравнивается с хешем, сохранённым в базе данных, с помощью функции `password_verify`. После успешного входа создаётся сессия, в которую сохраняется идентификатор пользователя. Защита служебных страниц реализована в базовом контроллере: метод `requireAdmin()` проверяет авторизацию и наличие роли `admin`, иначе перенаправляет на страницу входа.

The screenshot shows a web browser displaying the login page for the 'Лапки' (Paws) website. The page has a light beige background. At the top, there is a navigation bar with the site logo 'Лапки' on the left and links for 'Главная', 'Питомцы', 'Статьи', 'Калькулятор', 'Контакты', and a 'Вход' button on the right. The main content area features a white login form with a lock icon and the title 'Вход для сотрудников' (Login for staff), with the subtitle 'Панель управления приютом' (Shelter management panel). The form includes an 'E-mail' field with the text 'admin@lapki.ru', a 'Пароль' (Password) field with masked characters, and an orange 'Войти' (Login) button. Below the button, a text box shows 'Демо-доступ: admin@lapki.ru / admin123'. The footer is dark brown and contains the site logo, a brief description of the shelter's mission, a list of sections ('Разделы') including 'Наши питомцы', 'Статьи и советы', 'Калькулятор корма', and 'Связаться с нами', and a list of contacts ('Контакты') including 'Хочу забрать питомца', 'Стать волонтером', 'Сервисная информация', and 'Вход для сотрудников'. A copyright notice at the bottom reads: '© 2026 Приют «Лапки». Курсовой проект по дисциплине «Серверная веб-разработка».

Рисунок 2. Форма входа в административную панель

После входа администратор попадает на панель управления (рисунок 3) со сводной статистикой и быстрыми ссылками на основные разделы.

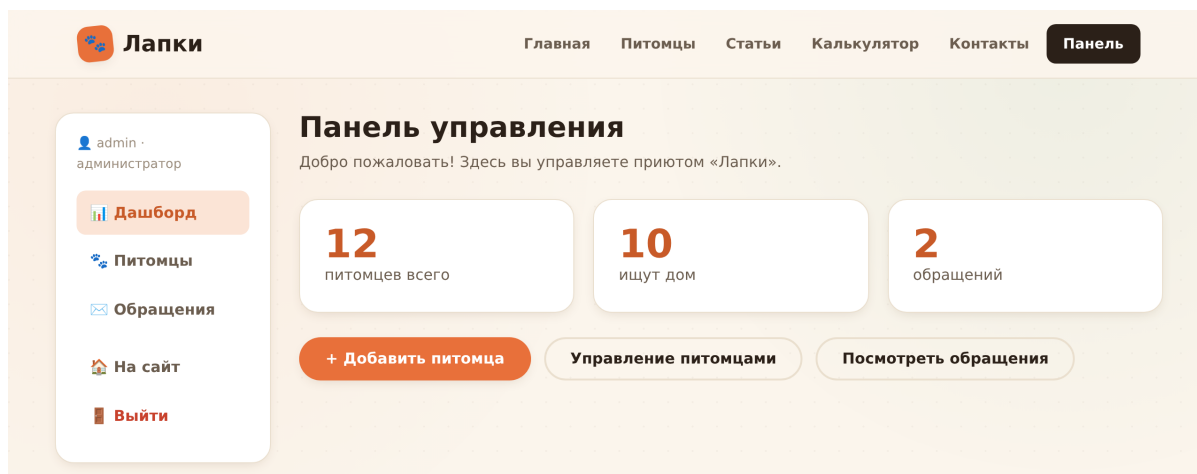


Рисунок 3. Панель управления (дашборд)

Ключевым элементом администрирования является управление карточками питомцев — оно полностью соответствует функциональности, изученной при разработке «записной книжки» в предыдущих лабораторных работах. Список выводится в виде таблицы с постраничной навигацией (по 10 записей) и сортировкой по дате добавления, кличке, возрасту и статусу (рисунок 4).

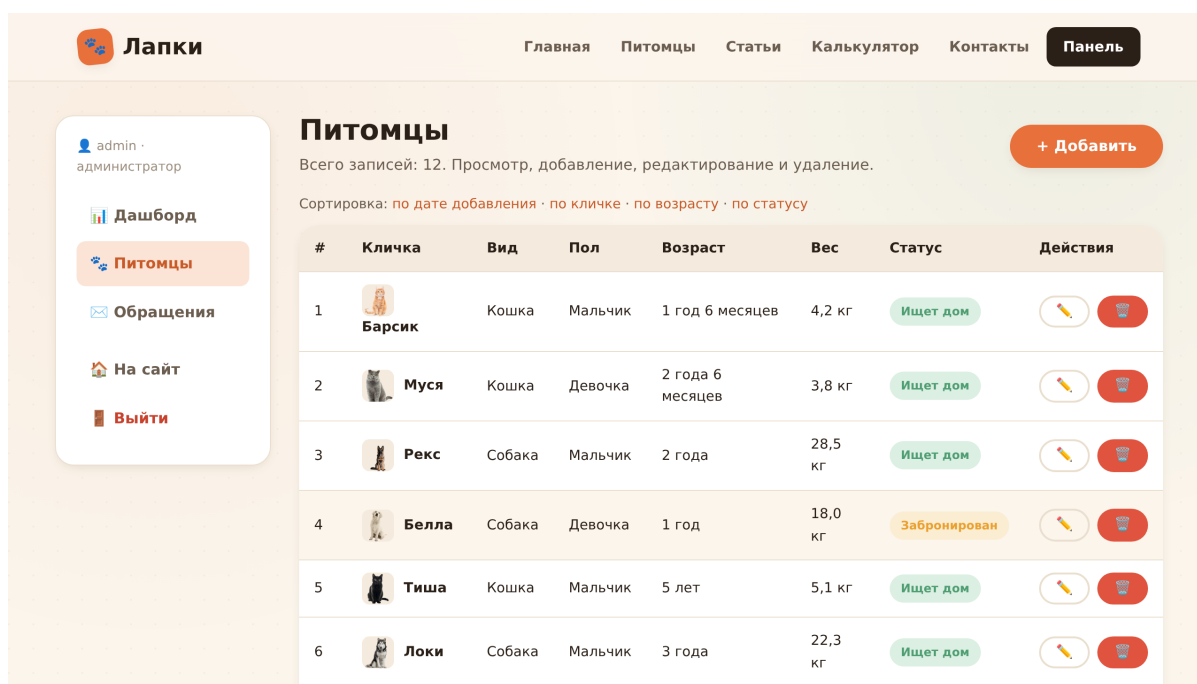


Рисунок 4. Список питомцев в админ-панели: сортировка и пагинация

Добавление и редактирование выполняются через единую форму (рисунок 5) с серверной валидацией. Сохранение реализовано единым методом `save()`

базового класса, который сам выбирает операцию INSERT или UPDATE в зависимости от того, существует ли уже идентификатор записи.

The screenshot shows the 'Лапки' website admin panel. The top navigation bar includes links for 'Главная', 'Питомцы', 'Статьи', 'Калькулятор', 'Контакты', and 'Панель'. The left sidebar shows the user 'admin · администратор' and navigation links for 'Дашборд', 'Питомцы' (highlighted), 'Обращения', 'На сайт', and 'Выйти'. The main content area is titled '← К списку питомцев' and 'Редактировать питомца'. The form contains the following fields:

- Кличка ***: Text input with value 'Барсик'.
- Фото питомца**: Text input with value '/uploads/barsik.png'. Below it, a note states: 'Ссылка на фото (URL) или путь к файлу. Можно оставить пустым — тогда подберётся фото по виду животного. Допустим и эмодзи (🐾)'.
- Вид ***: Dropdown menu with value 'Кошка'.
- Порода**: Text input with value 'Беспородный'.
- Пол ***: Dropdown menu with value 'Мальчик'.
- Окрас**: Text input with value 'Рыжий'.
- Возраст (месяцев)**: Text input with value '18'.
- Вес (кг)**: Text input with value '4.2'.
- Активность**: Dropdown menu with value 'Средняя'.
- Статус**: Dropdown menu with value 'Ищет дом'.
- Описание ***: Text area with value 'Ласковый рыжий кот, обожает спать на коленях и тёплые батареи. Приучен к лотку.'

Рисунок 5. Форма добавления и редактирования карточки питомца

Аналогичный CRUD реализован для статей блога (рисунок 6). Создание статьи автоматически назначает её автором текущего администратора, благодаря чему на публичной странице статьи корректно отображается имя автора (демонстрация требования ДЗ 5 из третьей лабораторной работы).

The screenshot shows the 'Лапки' website admin panel. The top navigation bar includes links for 'Главная', 'Питомцы', 'Статьи' (highlighted), 'Калькулятор', 'Контакты', and 'Панель'. The left sidebar shows the user 'admin · администратор' and navigation links for 'Дашборд', 'Питомцы', 'Статьи' (highlighted), 'Обращения', 'На сайт', and 'Выйти'. The main content area is titled 'Статьи' and includes a '+ Добавить статью' button. Below the title, it says 'Всего статей: 4. Добавление, редактирование и удаление.'.

#	Заголовок	Дата	Действия
4	Адаптация животного в новом доме	27.05.2026	
3	Первая прогулка с собакой из приюта	27.05.2026	
2	Чем кормить кошку: основы здорового рациона	27.05.2026	
1	Как подготовить дом к появлению питомца	27.05.2026	

Рисунок 6. Управление статьями в административной панели

Обращения, поступившие через форму обратной связи, доступны администратору в отдельном разделе (рисунок 7), где видны имя отправителя, тип обращения, текст и выбранные способы связи.

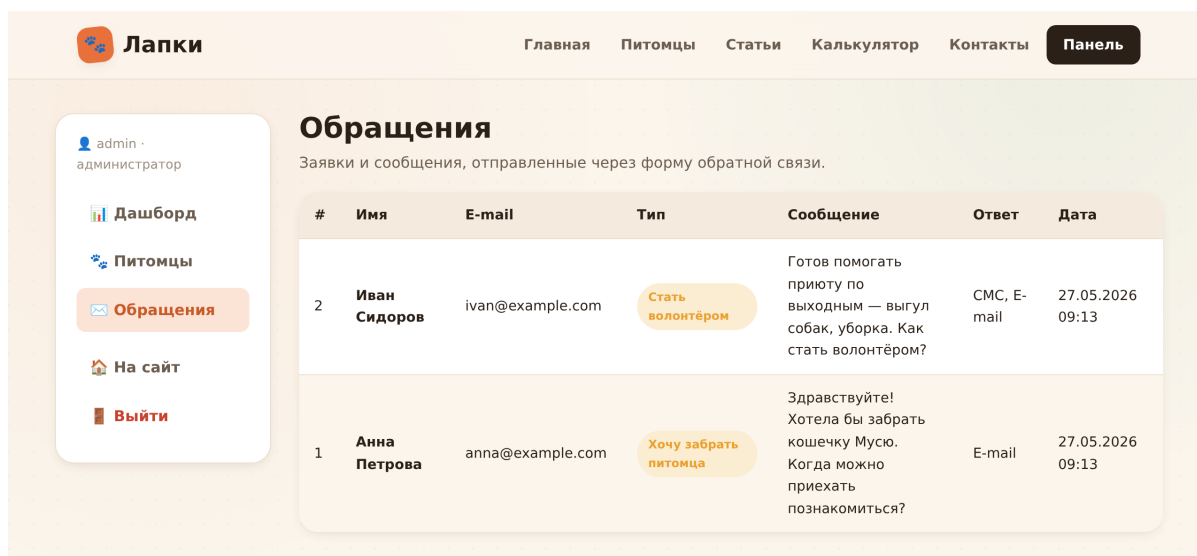


Рисунок 7. Раздел обращений в административной панели

2.4 Главная и детальные страницы

Главная страница (рисунок 8) выполняет роль витрины приюта: приветственный блок с призывом к действию и статистикой, подборка питомцев, ищущих дом, лента последних статей и итоговый призыв стать волонтером. Все элементы оформлены в тёплой дружелюбной палитре с использованием скруглённых карточек, мягких теней и адаптивной сетки.

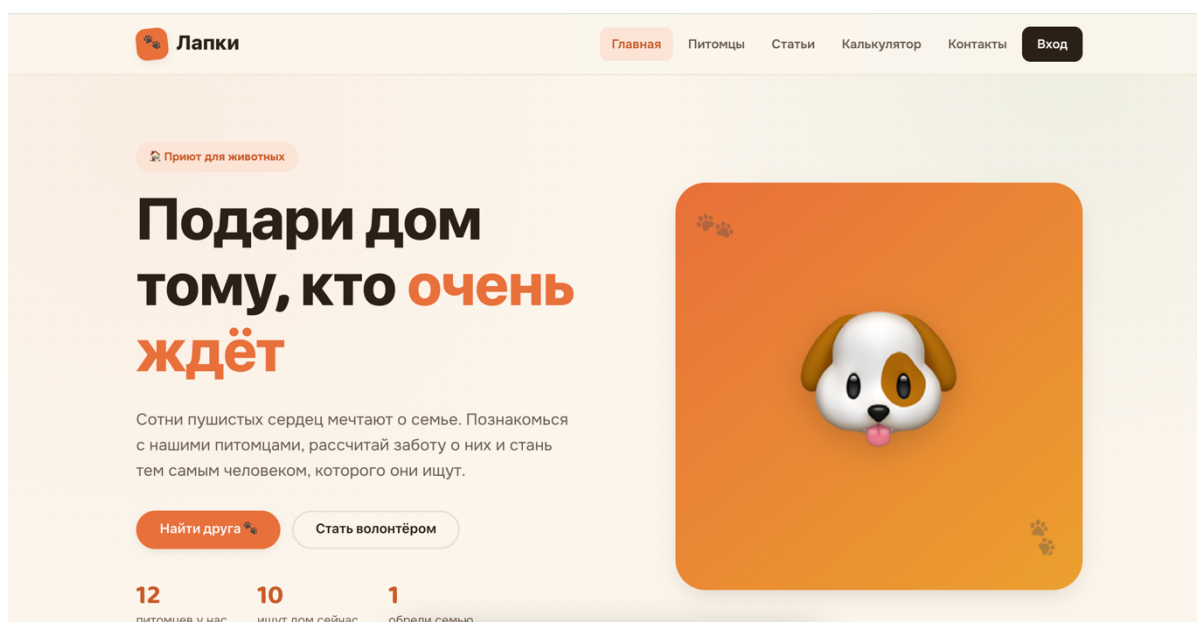


Рисунок 8. Главная страница сайта

Каталог питомцев (рисунок 9) поддерживает фильтрацию по виду животного через GET-параметр. Каждая карточка отображает реальную фотографию питомца на тёплом градиентном фоне, кличку, основные характеристики и статус («Ищет дом», «Забронирован», «Дома»).

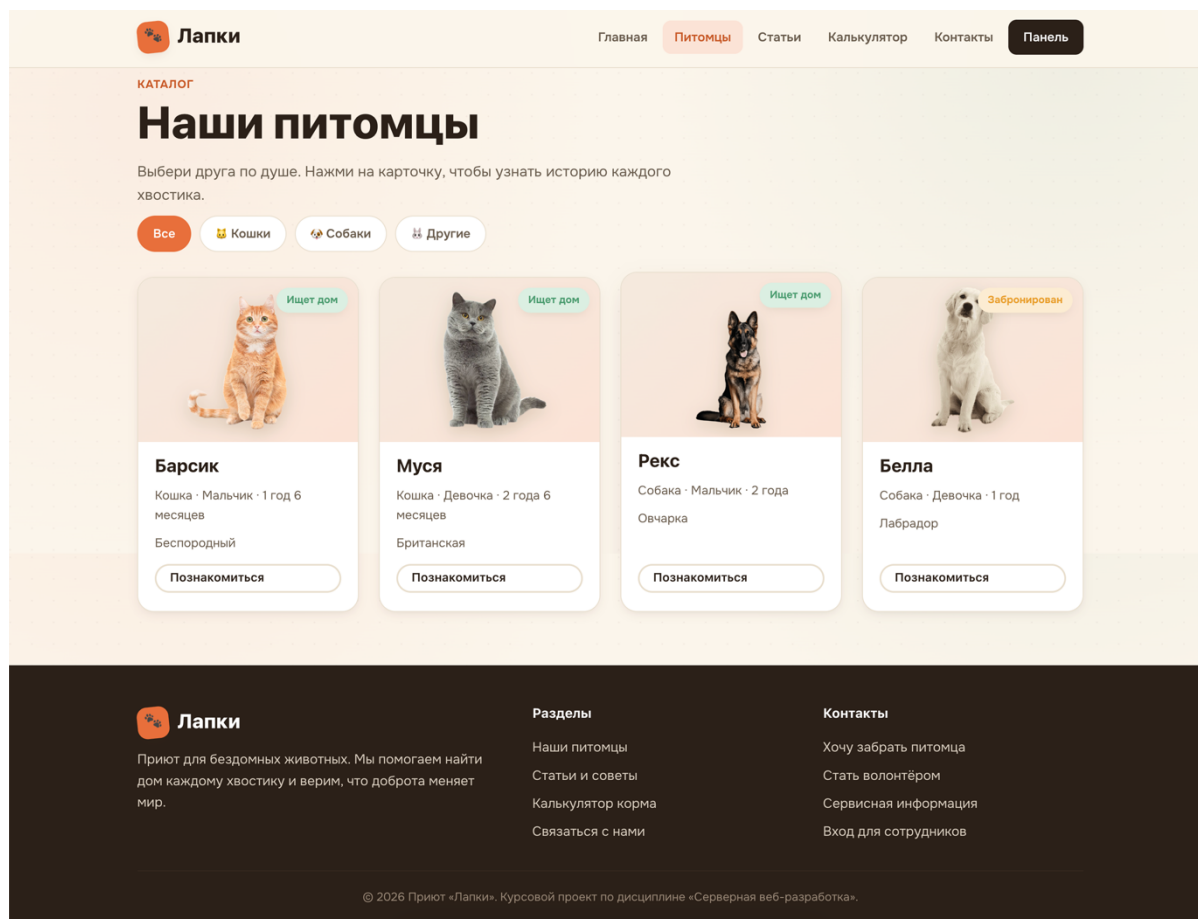


Рисунок 9. Каталог питомцев с фильтрацией

Детальная страница питомца (рисунок 10) выводит подробное описание, таблицу характеристик и кнопки действий, включая переход к калькулятору корма с заранее подставленными параметрами животного. Это создаёт связную пользовательскую историю: посетитель не просто читает анкету, но сразу получает практическую информацию о содержании.

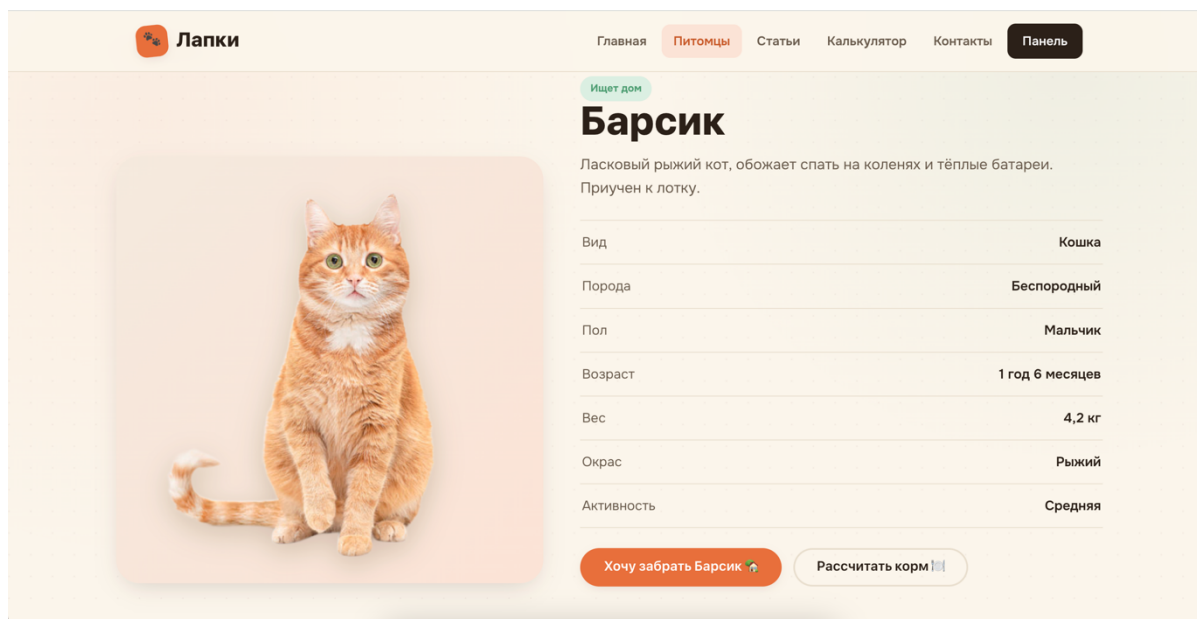


Рисунок 10. Детальная страница питомца

Раздел статей реализует функциональность блога. На детальной странице статьи (рисунок 11) после получения материала выполняется дополнительный запрос к таблице users для получения автора, и его никнейм выводится в шаблоне. Заголовок вкладки браузера (title) формируется динамически через переменную шаблона.

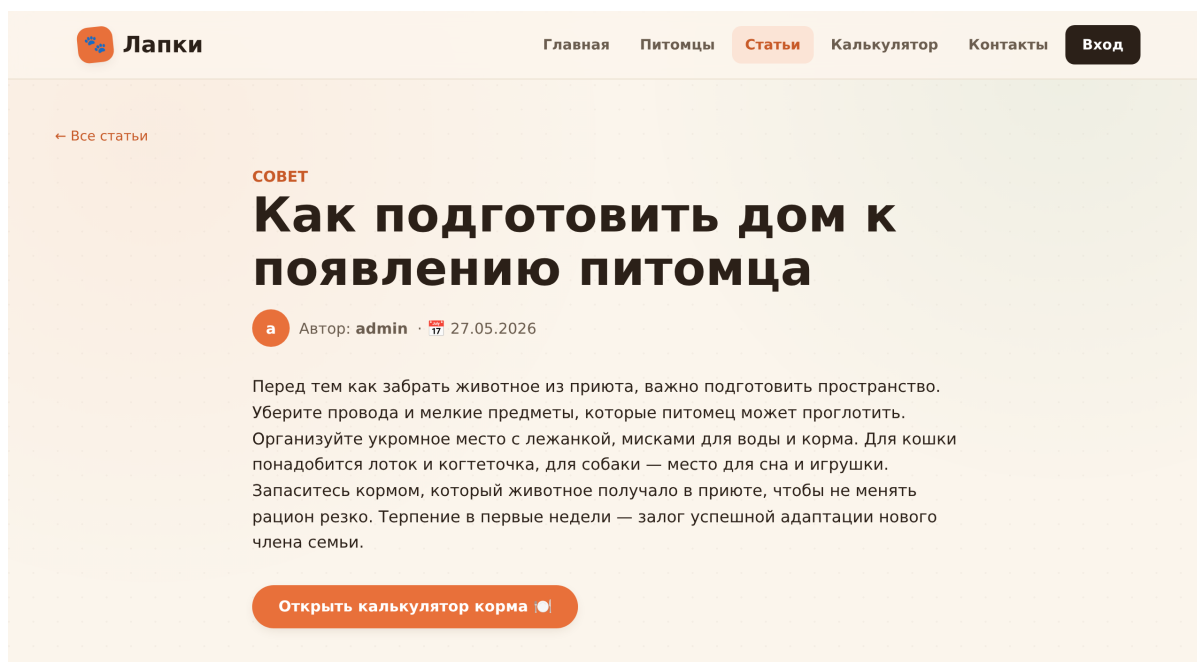


Рисунок 11. Детальная страница статьи с указанием автора

2.5 Дополнительный функционал

Калькулятор суточной нормы корма (рисунок 12) реализует расчёт данных на основе параметров, введённых пользователем: вида животного, веса в килограммах, возрастной группы и уровня активности. Расчёт выполняется на стороне сервера: базовая норма (20 г на килограмм для кошек, 25 г — для собак) умножается на коэффициенты возраста и активности. Дополнительно рассчитываются примерная калорийность, рекомендованное число кормлений и расход корма в месяц.

The screenshot shows a web application for calculating pet food requirements. The header includes the site name 'Лапки' and navigation links: Главная, Питомцы, Статьи, Калькулятор (highlighted), Контакты, and a login button Вход. The main heading is 'Калькулятор корма' with a subtitle 'Динамический расчёт'. Below the heading, a brief description states: 'Рассчитайте суточную норму сухого корма для питомца на основе его параметров: вида, веса, возраста и активности.' The form contains four input sections: 'Вид животного' (Cat selected), 'Вес питомца, кг' (4.5 kg), 'Возрастная группа' (Adult selected), and 'Активность' (Average selected). To the right, a dark box displays the results: 'Суточная норма корма' (Daily food norm) of 90 g/day. Below this, four boxes show: 315 kcal/day, 3 feedings/day, 30 g per feeding, and approximately 2.7 kg per month. A disclaimer at the bottom of the results box states: '* Расчёт ориентировочный. Точную норму уточняйте у ветеринара и на упаковке корма.'

Рисунок 12. Калькулятор нормы корма с подробным результатом

```
$basePerKg = $species === 'cat' ? 20 : 25;  
$ageFactor = ['baby'=>1.6, 'adult'=>1.0, 'senior'=>0.85][$age];  
$activityFactor = ['low'=>0.9, 'normal'=>1.0, 'high'=>1.2][$activity];  
$gramsPerDay = round($weight * $basePerKg * $ageFactor * $activityFactor);
```

Листинг 4. Алгоритм расчёта суточной нормы корма

Форма обратной связи (рисунок 13) содержит поля имени, e-mail, типа обращения, текста сообщения и флажки выбора способа ответа (СМС, e-mail). Данные отправляются методом POST, проходят серверную валидацию и сохраняются в базу данных через подготовленное выражение. После успешной отправки применяется паттерн «POST-Redirect-GET» и выводится сообщение об успехе.

The screenshot shows the 'Летки' website with a navigation bar containing links: Главная, Питомцы, Статьи, Калькулятор, Контакты, and a Вход button. The 'Контакты' section is active, displaying the heading 'Связаться с нами'. Below the heading is a subtext: 'Хотите забрать питомца, стать волонтером или просто задать вопрос? Напишите нам — мы ответим как можно скорее.' The contact form itself is a white card with the following fields: 'Имя' (Name) with a placeholder 'Как к вам обращаться?'; 'E-mail' with a placeholder 'you@example.com'; 'Тип обращения' (Type of request) as a dropdown menu with 'Выберите...'; 'Текст обращения' (Text of request) as a large text area with a placeholder 'Расскажите подробнее...'; and a section 'Как вам удобнее получить ответ?' (How would you prefer to receive an answer?) with two checkboxes: 'СМС' and 'E-mail'.

Рисунок 13. Форма обратной связи

Для демонстрации работы с протоколом HTTP реализована служебная страница (рисунок 14), выводящая заголовки ответа сервера, полученные с помощью функции `get_headers`. Это прямое наследование функциональности из первой лабораторной работы.

The screenshot shows the 'Летки' website with a navigation bar. The 'СЕРВИСНАЯ ИНФОРМАЦИЯ' (Service Information) section is active, displaying the heading 'HTTP-заголовки'. Below the heading is a subtext: 'Демонстрация работы функции `get_headers()` — вывод заголовков HTTP-ответа для указанного URL.' The form consists of a 'URL для запроса' (URL for request) input field containing 'https://mospolytech.ru' and a 'Запросить' (Request) button. Below the form, a dark box displays the output of the `get_headers()` function for the specified URL: 'URL: https://mospolytech.ru', 'HTTP/1.1 403 Forbidden', 'x-deny-reason: host_not_allowed', 'content-length: 21', 'content-type: text/plain', 'date: Wed, 27 May 2026 09:13:30 GMT', and 'connection: close'.

Рисунок 14. Вывод HTTP-заголовков функцией `get_headers`

Демонстрационные роуты `/hello/<имя>` и `/bye/<имя>`, реализованные при выполнении третьей лабораторной работы, сохранены в проекте и доступны посетителю напрямую (рисунок 15).

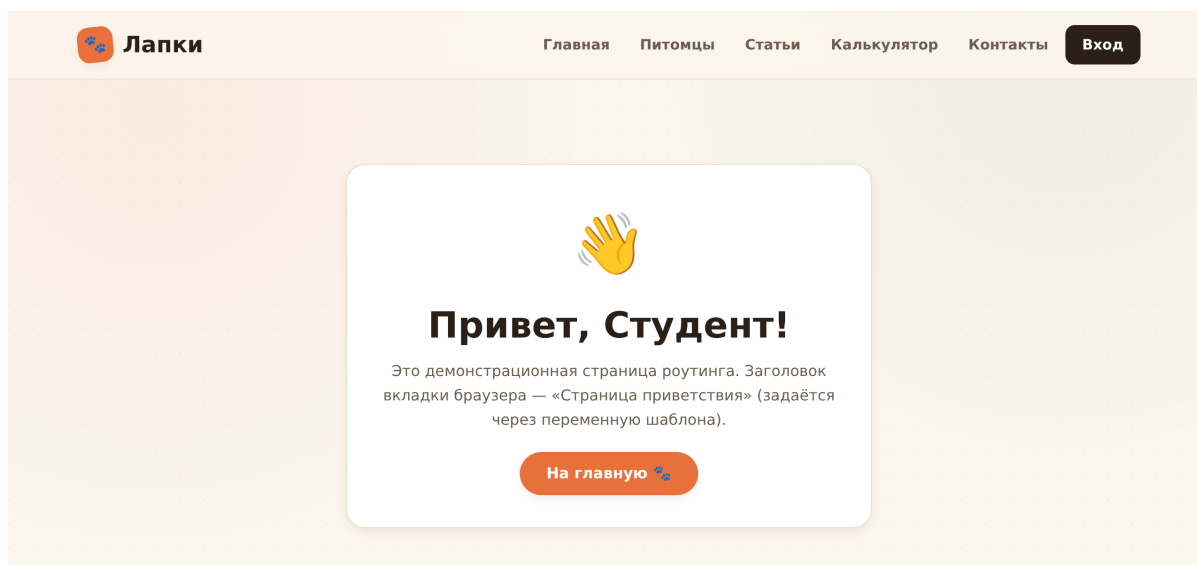


Рисунок 15. Демонстрационный роут `/hello/<имя>` с динамическим заголовком

Запросы на несуществующие адреса корректно обрабатываются: пользователю показывается оформленная в общем стиле страница 404 (рисунок 16) с кодом HTTP-ответа 404.

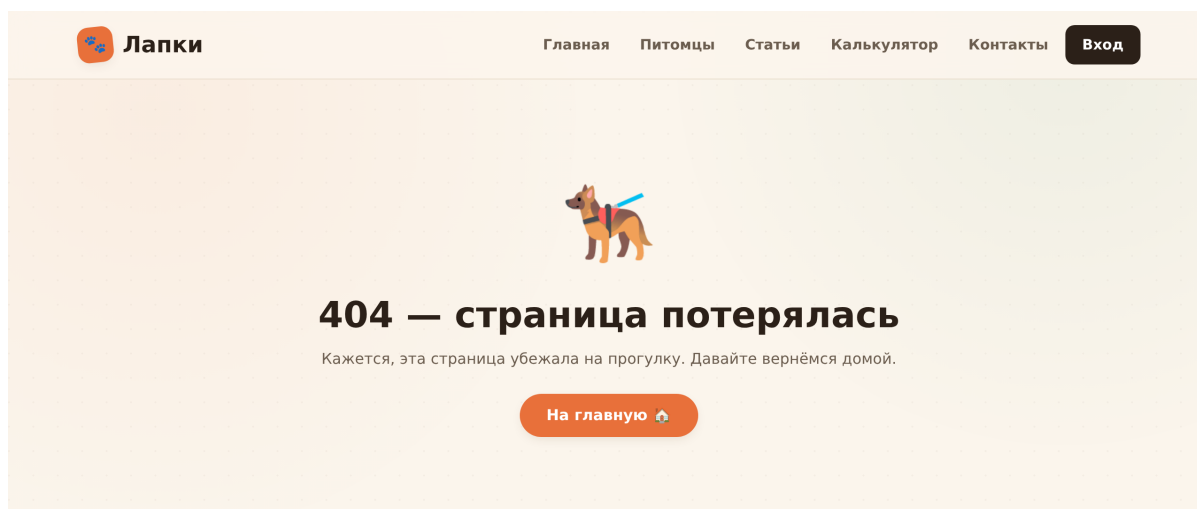


Рисунок 16. Страница ошибки 404

2.6 Безопасность

При проектировании учтены типовые требования безопасности веб-приложений.

Защита от SQL-инъекций. Все запросы к базе данных выполняются через подготовленные выражения PDO с именованными параметрами. Пользовательский ввод никогда не подставляется в SQL-строку напрямую.

Хранение паролей. Пароли пользователей хранятся в виде хешей, полученных функцией `password_hash` (алгоритм `bcrypt`). При входе используется `password_verify`; обратное восстановление пароля по хешу вычислительно невозможно.

Защита от XSS. Все пользовательские данные перед выводом в шаблонах экранируются функцией `htmlspecialchars`. Это предотвращает интерпретацию пользовательских строк как HTML или JavaScript.

Разграничение доступа. Служебные действия защищены вызовом `requireAdmin()` в начале метода. Гость, обратившийся к адресу административной части, перенаправляется на страницу входа.

Запрет прямого доступа к исходному коду. В каталоги `src/` и `templates/`, расположенные в корне сайта, помещены файлы `.htaccess` с правилом `Require all denied`. Это блокирует попытку прочитать исходный код через HTTP — все обращения должны идти через фронт-контроллер.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта разработан полнофункциональный веб-сайт приюта для животных «Лапки» на основе собственного backend-фреймворка на языке PHP. Проект объединил в единый целостный продукт все технологии, изученные в течение курса «Серверная веб-разработка».

Поставленные во введении задачи решены полностью. Проведён анализ предметной области и аналогичных решений, обоснован выбор технологий и архитектурных паттернов. Спроектирована и реализована база данных из четырёх связанных таблиц. Разработана архитектура приложения по паттерну MVC с собственным фронт-контроллером, маршрутизацией, базовым классом ActiveRecordEntity и компонентом представления.

Реализована административная часть с авторизацией по хешированному паролю, разграничением доступа по роли и полным набором CRUD-операций над двумя сущностями — питомцами и статьями. Списки выводятся с постраничной навигацией и сортировкой, добавление и редактирование выполняются через формы с серверной валидацией.

Разработаны главная и детальные страницы публичной части. В дополнение к базовым требованиям реализован интерактивный калькулятор нормы корма, демонстрирующий расчёт данных на основе параметров. Сохранены и интегрированы в проект функции из предыдущих работ: форма обратной связи, CRUD «записной книжки» в виде управления питомцами, маршрутизация и динамический заголовок.

Отдельное внимание уделено качеству пользовательского интерфейса: сайт имеет современный адаптивный дизайн в дружелюбной тематической палитре, удобную навигацию и продуманные сценарии взаимодействия с реальными фотографиями питомцев.

Перспективы развития проекта включают: систему пожертвований и подписки на новости, личный кабинет волонтера с историей помощи,

многоязычность интерфейса и интеграцию с социальными сетями для повышения охвата.

Цель курсового проекта достигнута. Проект пригоден для дальнейшего развития и адаптации под нужды реального приюта.

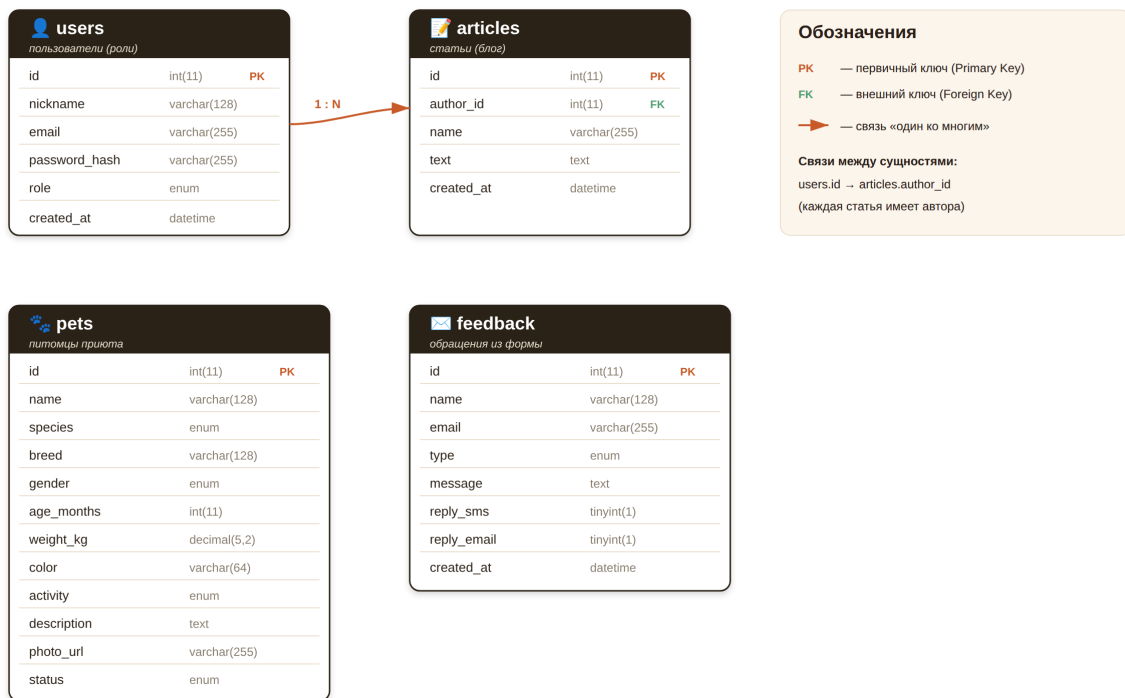
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

14. Котеров Д. В. PHP 8 / Д. В. Котеров, И. В. Симдянов. — Санкт-Петербург : БХВ-Петербург, 2023. — 1264 с. — ISBN 978-5-9775-6841-5. — Текст : непосредственный.
15. Никсон Р. Создаём динамические веб-сайты с помощью PHP, MySQL, JavaScript, CSS и HTML5 / Р. Никсон. — 5-е изд. — Санкт-Петербург : Питер, 2021. — 816 с. — ISBN 978-5-4461-1675-1. — Текст : непосредственный.
16. Фримен Э. Паттерны проектирования / Э. Фримен, Э. Робсон. — 2-е изд. — Санкт-Петербург : Питер, 2022. — 656 с. — ISBN 978-5-4461-1819-9. — Текст : непосредственный.
17. Дронов В. А. Самоучитель: HTML, CSS и JavaScript / В. А. Дронов. — Санкт-Петербург : БХВ-Петербург, 2021. — 768 с. — ISBN 978-5-9775-6644-2. — Текст : непосредственный.
18. Документация языка PHP. — URL: <https://www.php.net/docs.php> (дата обращения: 25.05.2026). — Текст : электронный.
19. Документация СУБД MySQL. — URL: <https://dev.mysql.com/doc/> (дата обращения: 25.05.2026). — Текст : электронный.
20. PHP Data Objects (PDO). — URL: <https://www.php.net/manual/ru/book.pdo.php> (дата обращения: 25.05.2026). — Текст : электронный.
21. MDN Web Docs: HTML, CSS, HTTP. — URL: <https://developer.mozilla.org/ru/> (дата обращения: 26.05.2026). — Текст : электронный.
22. Документация Apache HTTP Server: mod_rewrite. — URL: https://httpd.apache.org/docs/current/mod/mod_rewrite.html (дата обращения: 26.05.2026). — Текст : электронный.

ПРИЛОЖЕНИЕ А

Структура базы данных

В приложении приведена ER-диаграмма базы данных проекта (рисунок А.1), подробное описание полей каждой из четырёх таблиц и полный SQL-листинг операторов создания таблиц с тестовыми данными.



ER-диаграмма базы данных приюта «Лапки»

Рисунок А.1. ER-диаграмма базы данных приюта «Лапки»

А.1 Таблица users — пользователи

Хранит учётные записи системы. Пароль хранится в виде хеша. Поле role определяет уровень доступа (admin/user); только пользователи с ролью admin получают доступ к административной части.

Таблица А.1. Поля сущности users

Поле	Тип	Ключ	Описание
id	INT, AUTO_INCREMENT	PK	Уникальный идентификатор пользователя
nickname	VARCHAR(128)	UQ	Псевдоним для отображения (выводится как автор статьи)
email	VARCHAR(255)	UQ	Адрес электронной почты для входа в систему
password_hash	VARCHAR(255)		Хеш пароля, полученный функцией password_hash (bcrypt)

role	ENUM('admin','user')		Роль пользователя; определяет уровень доступа
created_at	DATETIME		Момент создания учётной записи

A.2 Таблица pets — питомцы

Центральная сущность витрины и одновременно объект CRUD-операций в админ-панели. Структура повторяет логику «записной книжки» из второй лабораторной работы: множество атрибутов одной карточки.

Таблица A.2. Поля сущности pets

Поле	Тип	Ключ	Описание
id	INT, AUTO_INCREMENT	PK	Уникальный идентификатор питомца
name	VARCHAR(128)		Кличка
species	ENUM('cat','dog','other')		Вид животного
breed	VARCHAR(128)		Порода (может быть пустым для беспородных)
gender	ENUM('male','female')		Пол питомца
age_months	INT(11)		Возраст в месяцах (для удобства расчётов)
weight_kg	DECIMAL(5,2)		Вес в килограммах (используется в калькуляторе корма)
color	VARCHAR(64)		Окрас
activity	ENUM('low','normal','high')		Уровень активности (коэффициент для калькулятора корма)
description	TEXT		Свободный текст: характер, история, особенности
photo_url	VARCHAR(255)		Путь к фотографии (/uploads/...) или внешний URL
status	ENUM('available','reserved','adopted')		Статус: ищет дом / забронирован / нашёл дом
created_at	DATETIME		Момент добавления записи

A.3 Таблица articles — статьи блога

Содержит материалы блога приюта. Связана с таблицей users внешним ключом author_id, что соответствует требованию ДЗ 5 третьей лабораторной работы (вывод автора статьи).

Таблица A.3. Поля сущности articles

Поле	Тип	Ключ	Описание
------	-----	------	----------

id	INT, AUTO_INCREMENT	PK	Уникальный идентификатор статьи
author_id	INT(11)	FK	Внешний ключ → users.id; автор статьи
name	VARCHAR(255)		Заголовок статьи
text	TEXT		Полный текст статьи (абзацы разделяются переводом строки)
created_at	DATETIME		Момент публикации

A.4 Таблица feedback — обращения

Сюда сохраняются данные, отправленные посетителями через форму обратной связи. Записи доступны администратору в соответствующем разделе панели.

Таблица A.4. Поля сущности feedback

Поле	Тип	Ключ	Описание
id	INT, AUTO_INCREMENT	PK	Уникальный идентификатор обращения
name	VARCHAR(128)		Имя отправителя
email	VARCHAR(255)		Адрес для ответа
type	ENUM('adopt','volunteer','question','gratitude')		Тип обращения
message	TEXT		Текст обращения
reply_sms	TINYINT(1)		Желание получить ответ СМС (флажок чекбокса)
reply_email	TINYINT(1)		Желание получить ответ по e-mail (флажок чекбокса)
created_at	DATETIME		Момент отправки

A.5 SQL-листинг создания таблиц

Полный SQL-скрипт создания базы данных и таблиц с тестовыми данными расположен в файле database.sql в корне проекта. Ниже приведены ключевые операторы CREATE TABLE.

```
-- =====
-- База данных для сайта приюта животных «Лапки»
-- Курсовой проект по дисциплине «Серверная веб-разработка»
-- =====

SET NAMES utf8mb4;

CREATE DATABASE IF NOT EXISTS `lapki`
  DEFAULT CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
```

```

USE `lapki`;

-- ===== Пользователи =====
-- Роль admin даёт доступ к панели управления.
DROP TABLE IF EXISTS `users`;
CREATE TABLE `users` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `nickname` varchar(128) NOT NULL,
  `email` varchar(255) NOT NULL,
  `password_hash` varchar(255) NOT NULL,
  `role` enum('admin','user') NOT NULL DEFAULT 'user',
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `nickname` (`nickname`),
  UNIQUE KEY `email` (`email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Пароль администратора: admin123 (хранится в виде password_hash)
INSERT INTO `users` (`nickname`, `email`, `password_hash`, `role`) VALUES
('admin', 'admin@lapki.ru', '$2y$10$ty9iw4hnztYpcXDsk6mOWOwx1YYeZfD3i2kkYTLer2VQYMTw6N39q', 'admin'),
('volunteer', 'vol@lapki.ru', '$2y$10$ty9iw4hnztYpcXDsk6mOWOwx1YYeZfD3i2kkYTLer2VQYMTw6N39q', 'user');

-- ===== Питомцы =====
-- Это и каталог на витрине, и таблица для CRUD в админке
-- (по аналогии с «записной книжкой»: множество полей-атрибутов).
DROP TABLE IF EXISTS `pets`;
CREATE TABLE `pets` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `name` varchar(128) NOT NULL, -- кличка
  `species` enum('cat','dog','other') NOT NULL, -- вид
  `breed` varchar(128) NOT NULL DEFAULT '', -- порода
  `gender` enum('male','female') NOT NULL, -- пол
  `age_months` int(11) NOT NULL DEFAULT 0, -- возраст в месяцах
  `weight_kg` decimal(5,2) NOT NULL DEFAULT 0.00, -- вес, кг
  `color` varchar(64) NOT NULL DEFAULT '', -- окрас
  `activity` enum('low','normal','high') NOT NULL DEFAULT 'normal', -- активность
  `description` text NOT NULL, -- описание
  `photo_url` varchar(255) NOT NULL DEFAULT '', -- ссылка на фото
  (эмодзи/url)
  `status` enum('available','reserved','adopted') NOT NULL DEFAULT 'available',
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

INSERT INTO `pets`
(`name`, `species`, `breed`, `gender`, `age_months`, `weight_kg`, `color`, `activity`, `description`, `photo_url`, `status`) VALUES
('Барсик', 'cat', 'Беспородный', 'male', 18, 4.20, 'Рыжий', 'normal', 'Ласковый рыжий кот, обожает спать на коленях и тёплые батареи. Приучен к лотку.', '/uploads/barsik.png', 'available'),
('Муся', 'cat', 'Британская', 'female', 30, 3.80, 'Серый', 'low', 'Спокойная домоседка, идеальна для тихой семьи. Любит наблюдать за птицами из окна.', '/uploads/musya.png', 'available'),
('Рекс', 'dog', 'Овчарка', 'male', 24, 28.50, 'Чепрачный', 'high', 'Энергичный и верный пёс, прошёл базовую дрессировку. Нужен активный хозяин и простор.', '/uploads/reks.png', 'available'),
('Белла', 'dog', 'Лабрадор', 'female', 12, 18.00, 'Палевый', 'high', 'Дружелюбный щенок-подросток, обожает детей и игры с мячом. Очень умная.', '/uploads/bella.png', 'reserved'),
('Тиша', 'cat', 'Беспородный', 'male', 60, 5.10, 'Чёрный', 'low', 'Солидный взрослый кот с характером, любит уют и размеренную жизнь. Подойдёт спокойному дому.', '/uploads/tisha.png', 'available'),
('Локи', 'dog', 'Хаски', 'male', 36, 22.30, 'Чёрно-белый', 'high', 'Невероятно активный хаски, нужен опытный хозяин и долгие прогулки. Обожают бегать.', '/uploads/loki.png', 'available'),
('Соня', 'cat', 'Сиамская', 'female', 8, 2.60, 'Колор-пойнт', 'normal', 'Игривая молодая кошечка, очень общительная и любопытная. Быстро привыкает к людям.', '/uploads/sonya.png', 'available'),
('Граф', 'dog', 'Такса', 'male', 48, 7.40, 'Рыжий', 'normal', 'Умный и преданный таксик, отлично ладит с другими животными. Любит длинные прогулки.', '/uploads/graf.png', 'adopted'),

```

```

('Дымка','cat','Беспородная','female',15,3.30,'Дымчатый','normal','Нежная и тихая
кошка, очень привязывается к хозяину. Мечтает о любящем
доме.','/uploads/dymka.png','available'),
('Бади','dog','Бигль','male',20,12.10,'Трёхцветный','high','Весёлый и любознательный
бигль, обожает нюхать всё вокруг. Нужны активные
прогулки.','/uploads/badi.png','available'),
('Тесс','other','Карликовый хотот','female',48,1.40,'Белый с
серым','normal','Очаровательная крольчиха карликовой породы хотот. Спокойная,
чистоплотная и контактная. Будет рада уютному дому и свежей
зелени.','/uploads/tess.png','available'),
('Пушок','cat','Мейн-кун','male',40,7.80,'Мраморный','normal','Огромный добродушный
мейн-кун, ласковый гигант. Прекрасно ладит со
всеми.','/uploads/pushok.png','available');

-- ===== Статьи (блог) =====
-- Связь author_id -> users.id
DROP TABLE IF EXISTS `articles`;
CREATE TABLE `articles` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `author_id` int(11) NOT NULL,
  `name` varchar(255) NOT NULL,
  `text` text NOT NULL,
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`)

```

Листинг 5. Фрагмент файла database.sql: создание таблиц users и pets

ПРИЛОЖЕНИЕ Б

Листинги ключевых модулей исходного кода

В приложении приведены полные листинги ключевых файлов исходного кода проекта. Полный код доступен в репозитории, указанном во введении. Имя файла указано перед каждым листингом.

index.php

```
<?php

// Сессию стартуем в самом начале, до любого вывода —
// это гарантирует надёжное сохранение авторизации между запросами.
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

try {
    spl_autoload_register(function (string $className) {
        $className = str_replace('\\', '/', $className);
        require_once __DIR__ . '/src/' . $className . '.php';
    });

    $route = $_GET['route'] ?? '';
    $routes = require __DIR__ . '/src/routes.php';

    $isRouteFound = false;
    foreach ($routes as $pattern => $controllerAndAction) {
        preg_match($pattern, $route, $matches);
        if (!empty($matches)) {
            $isRouteFound = true;
            break;
        }
    }

    if (!$isRouteFound) {
        throw new \MyProject\Exceptions\NotFoundException();
    }

    unset($matches[0]);

    $controllerName = $controllerAndAction[0];
    $actionName = $controllerAndAction[1];

    $controller = new $controllerName();
    $controller->$actionName(...$matches);
} catch (\MyProject\Exceptions\NotFoundException $e) {
    $view = new \MyProject\View\View(__DIR__ . '/templates');
    $view->renderHtml('errors/404.php', ['title' => 'Страница не найдена',
    'currentUser' => null], 404);
} catch (\MyProject\Exceptions\DbException $e) {
    $view = new \MyProject\View\View(__DIR__ . '/templates');
    $view->renderHtml('errors/500.php', ['title' => 'Ошибка сервера', 'currentUser' =>
    null, 'error' => $e->getMessage()], 500);
}
```

Листинг Б.1. Фронт-контроллер (точка входа в приложение)

src/routes.php

```
<?php

use MyProject\Controllers\MainController;
use MyProject\Controllers\PetsController;
use MyProject\Controllers\ArticlesController;
use MyProject\Controllers\FeedbackController;
use MyProject\Controllers\CalculatorController;
use MyProject\Controllers\AdminController;

return [
    // ----- Публичная часть -----
    '^pets$~' => [PetsController::class, 'list'],
    '^pets/(\d+)$~' => [PetsController::class, 'show'],

    '^articles$~' => [ArticlesController::class, 'list'],
    '^articles/(\d+)$~' => [ArticlesController::class, 'show'],

    '^calculator$~' => [CalculatorController::class, 'feed'],

    '^feedback$~' => [FeedbackController::class, 'form'],
    '^feedback/submit$~' => [FeedbackController::class, 'submit'],

    '^headers$~' => [MainController::class, 'headers'],

    // Демонстрация роутинга (ДЗ3-4)
    '^hello/(.+)~' => [MainController::class, 'sayHello'],
    '^bye/(.+)~' => [MainController::class, 'sayBye'],

    // ----- Админ-панель -----
    '^admin/login$~' => [AdminController::class, 'loginForm'],
    '^admin/login-submit$~' => [AdminController::class, 'login'],
    '^admin/logout$~' => [AdminController::class, 'logout'],

    '^admin/pets/add$~' => [AdminController::class, 'petAddForm'],
    '^admin/pets/store$~' => [AdminController::class, 'petAdd'],
    '^admin/pets/edit/(\d+)$~' => [AdminController::class, 'petEditForm'],
    '^admin/pets/update/(\d+)$~' => [AdminController::class, 'petEdit'],
    '^admin/pets/delete/(\d+)$~' => [AdminController::class, 'petDelete'],
    '^admin/pets$~' => [AdminController::class, 'petsList'],

    '^admin/articles/add$~' => [AdminController::class, 'articleAddForm'],
    '^admin/articles/store$~' => [AdminController::class, 'articleAdd'],
    '^admin/articles/edit/(\d+)$~' => [AdminController::class, 'articleEditForm'],
    '^admin/articles/update/(\d+)$~' => [AdminController::class, 'articleEdit'],
    '^admin/articles/delete/(\d+)$~' => [AdminController::class, 'articleDelete'],
    '^admin/articles$~' => [AdminController::class, 'articlesList'],

    '^admin/feedback$~' => [AdminController::class, 'feedbackList'],
    '^admin$~' => [AdminController::class, 'dashboard'],

    // ----- Главная -----
    '^$~' => [MainController::class, 'main'],
];
```

Листинг Б.2. Таблица маршрутов

src/MyProject/Services/Db.php

```
<?php

namespace MyProject\Services;

use MyProject\Exceptions\DbException;
```

```

class Db
{
    /** @var \PDO */
    private $pdo;

    public function __construct()
    {
        $dbOptions = (require __DIR__ . '/../../settings.php')['db'];

        try {
            $this->pdo = new \PDO(
                'mysql:host=' . $dbOptions['host'] . ';dbname=' . $dbOptions['dbname']
                . ';charset=utf8mb4',
                $dbOptions['user'],
                $dbOptions['password']
            );
            $this->pdo->setAttribute(\PDO::ATTR_ERRMODE, \PDO::ERRMODE_EXCEPTION);
            $this->pdo->exec("SET NAMES 'utf8mb4'");
        } catch (\PDOException $e) {
            throw new DbException($e->getMessage());
        }
    }

    /**
     * Выполняет SQL-запрос и возвращает результат в виде массива объектов.
     *
     * @param string $sql      Текст запроса (можно с псевдопеременными)
     * @param array  $params   Значения псевдопеременных
     * @param string $className Класс, объекты которого создавать из строк
     * @return array|null
     */
    public function query(string $sql, array $params = [], string $className =
'stdClass'): ?array
    {
        try {
            $sth = $this->pdo->prepare($sql);
            $result = $sth->execute($params);
            if (false === $result) {
                return null;
            }
            return $sth->fetchAll(\PDO::FETCH_CLASS, $className);
        } catch (\PDOException $e) {
            throw new DbException($e->getMessage());
        }
    }

    /**
     * Возвращает id последней вставленной строки.
     * Вызывается на том же экземпляре Db, что выполнял INSERT.
     */
    public function getLastInsertId(): int
    {
        return (int) $this->pdo->lastInsertId();
    }
}

```

Листинг Б.3. Сервис подключения к базе данных через PDO

src/MyProject/Services/Auth.php

```

<?php

namespace MyProject\Services;

use MyProject\Models\Users\User;

class Auth
{
    private static function startSession(): void

```

```

    {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
    }

    /** Пытается авторизовать пользователя по email и паролю. */
    public static function login(string $email, string $password): ?User
    {
        $user = User::findOneByColumn('email', $email);
        if ($user === null || !$user->checkPassword($password)) {
            return null;
        }
        self::startSession();
        $_SESSION['user_id'] = $user->getId();
        return $user;
    }

    public static function logout(): void
    {
        self::startSession();
        unset($_SESSION['user_id']);
    }

    /** Текущий авторизованный пользователь или null. */
    public static function getCurrentUser(): ?User
    {
        self::startSession();
        if (empty($_SESSION['user_id'])) {
            return null;
        }
        return User::getById((int) $_SESSION['user_id']);
    }

    public static function isAdmin(): bool
    {
        $user = self::getCurrentUser();
        return $user !== null && $user->isAdmin();
    }
}

```

Листинг Б.4. Сервис аутентификации (сессии, проверка пароля)

src/MyProject/Models/ActiveRecordEntity.php

```

<?php

namespace MyProject\Models;

use MyProject\Services\Db;

abstract class ActiveRecordEntity
{
    /** @var int */
    protected $id;

    public function getId(): int
    {
        return $this->id;
    }

    /**
     * Магический сеттер: PDO при FETCH_CLASS задаёт свойства по именам
     * столбцов (author_id), а у нас они в camelCase (authorId).
     */
    public function __set($name, $value)
    {
        $camelCaseName = $this->underscoreToCamelCase($name);
        $this->$camelCaseName = $value;
    }
}

```

```

    }

    private function underscoreToCamelCase(string $source): string
    {
        return lcfirst(str_replace(' ', '_', ucwords(str_replace('_', ' ', $source))));
    }

    private function camelCaseToUnderscore(string $source): string
    {
        return strtolower(preg_replace('/(?![^] [A-Z])/ ', '_$0', $source));
    }

    /**
     * Собирает ассоциативный массив [столбец => значение] из свойств объекта
     * с помощью рефлексии. Используется при сохранении.
     */
    private function mapPropertiesToDbFormat(): array
    {
        $reflector = new \ReflectionObject($this);
        $properties = $reflector->getProperties();

        $mapped = [];
        foreach ($properties as $property) {
            $propertyName = $property->getName();
            if ($propertyName === 'id') {
                continue;
            }
            $dbColumn = $this->camelCaseToUnderscore($propertyName);
            // пропускаем неинициализированные свойства
            if (isset($this->$propertyName)) {
                $mapped[$dbColumn] = $this->$propertyName;
            }
        }
        return $mapped;
    }

    /**
     * Сохраняет объект: если id есть — UPDATE, иначе — INSERT.
     */
    public function save(): void
    {
        $mappedProperties = $this->mapPropertiesToDbFormat();
        if ($this->id !== null) {
            $this->update($mappedProperties);
        } else {
            $this->insert($mappedProperties);
        }
    }

    private function insert(array $mappedProperties): void
    {
        $columns = [];
        $paramsNames = [];
        $params = [];
        foreach ($mappedProperties as $column => $value) {
            $columns[] = '`' . $column . '`';
            $paramName = ':' . $column;
            $paramsNames[] = $paramName;
            $params[$paramName] = $value;
        }

        $sql = 'INSERT INTO `' . static::getTableName() . '` (' . implode(', ',
$columns) . ') '
            . 'VALUES (' . implode(', ', $paramsNames) . ')';

        $db = new Db();
        $db->query($sql, $params, static::class);
        $this->id = $db->getLastInsertId();
    }

    private function update(array $mappedProperties): void
    {

```

```

        $columns = [];
        $params = ['id' => $this->id];
        foreach ($mappedProperties as $column => $value) {
            $columns[] = '`' . $column . '` = :' . $column;
            $params[':' . $column] = $value;
        }

        $sql = 'UPDATE `' . static::getTableName() . '` SET ' . implode(', ',
$columns)
            . ' WHERE id = :id;';

        $db = new Db();
        $db->query($sql, $params, static::class);
    }

    public function delete(): void
    {
        $db = new Db();
        $db->query(
            'DELETE FROM `' . static::getTableName() . '` WHERE id = :id;',
            ['id' => $this->id],
            static::class
        );
    }

    /** Все записи таблицы. */
    public static function findAll(): array
    {
        $db = new Db();
        return $db->query(
            'SELECT * FROM `' . static::getTableName() . '`;',
            [],
            static::class
        ) ?? [];
    }

    /** Одна запись по id или null. */
    public static function getById(int $id): ?self
    {
        $db = new Db();
        $entities = $db->query(
            'SELECT * FROM `' . static::getTableName() . '` WHERE id = :id;',
            ['id' => $id],
            static::class
        );
        return $entities ? $entities[0] : null;
    }

    /** Поиск одной записи по значению столбца (например, по email). */
    public static function findOneByColumn(string $columnName, $value): ?self
    {
        $db = new Db();
        $result = $db->query(
            'SELECT * FROM `' . static::getTableName() . '` WHERE `' . $columnName .
` = :value LIMIT 1;',
            ['value' => $value],
            static::class
        );
        return $result ? $result[0] : null;
    }

    /** Общее количество записей (для пагинации). */
    public static function countAll(): int
    {
        $db = new Db();
        $result = $db->query(
            'SELECT COUNT(*) AS cnt FROM `' . static::getTableName() . '`;'
        );
        return $result ? (int) $result[0]->cnt : 0;
    }

    /**
     * Записи с сортировкой и пагинацией.

```

```

        * $sortColumn — белый список проверяется в вызывающем коде.
        */
        public static function findSortedPaginated(string $sortColumn, int $limit, int
$offset): array
        {
            $db = new Db();
            $sql = 'SELECT * FROM `'. static::getTableName() . '` '
                . 'ORDER BY `'. $sortColumn . '` ASC '
                . 'LIMIT ' . (int) $limit . ' OFFSET ' . (int) $offset . ' ';
            return $db->query($sql, [], static::class) ?? [];
        }

        abstract protected static function getTableName(): string;
    }

```

Листинг Б.5. Базовый класс ORM (Active Record)

src/MyProject/Models/Pets/Pet.php

```

<?php

namespace MyProject\Models\Pets;

use MyProject\Models\ActiveRecordEntity;

class Pet extends ActiveRecordEntity
{
    /** @var string */
    protected $name;
    /** @var string */
    protected $species;
    /** @var string */
    protected $breed;
    /** @var string */
    protected $gender;
    /** @var int */
    protected $ageMonths;
    /** @var float */
    protected $weightKg;
    /** @var string */
    protected $color;
    /** @var string */
    protected $activity;
    /** @var string */
    protected $description;
    /** @var string */
    protected $photoUrl;
    /** @var string */
    protected $status;
    /** @var string */
    protected $createdAt;

    // ----- Геттеры -----
    public function getName(): string { return $this->name; }
    public function getSpecies(): string { return $this->species; }
    public function getBreed(): string { return $this->breed; }
    public function getGender(): string { return $this->gender; }
    public function getAgeMonths(): int { return (int) $this->ageMonths; }
    public function getWeightKg(): float { return (float) $this->weightKg; }
    public function getColor(): string { return $this->color; }
    public function getActivity(): string { return $this->activity; }
    public function getDescription(): string { return $this->description; }
    public function getPhotoUrl(): string { return $this->photoUrl; }
    public function getStatus(): string { return $this->status; }

    // ----- Сеттеры (для форм админки) -----
    public function setName(string $v): void { $this->name = $v; }
    public function setSpecies(string $v): void { $this->species = $v; }
    public function setBreed(string $v): void { $this->breed = $v; }
    public function setGender(string $v): void { $this->gender = $v; }
    public function setAgeMonths(int $v): void { $this->ageMonths = $v; }

```

```

public function setWeightKg(float $v): void { $this->weightKg = $v; }
public function setColor(string $v): void { $this->color = $v; }
public function setActivity(string $v): void { $this->activity = $v; }
public function setDescription(string $v): void { $this->description = $v; }
public function setPhotoUrl(string $v): void { $this->photoUrl = $v; }
public function setStatus(string $v): void { $this->status = $v; }

// ----- Фото питомца -----
/**
 * Возвращает URL настоящей фотографии питомца.
 * Если в photo_url задана ссылка (http...) или локальный файл (/uploads/..) –
 * используется она. Иначе подбирается настоящее фото по виду животного
 * (детерминированно по id) с бесплатного фотосервиса.
 */
public function getImageUrl(): string
{
    $val = trim((string) $this->photoUrl);
    if ($val !== '' && (strpos($val, 'http') === 0 || strpos($val, '/') === 0)) {
        return $val;
    }
    $id = $this->id ?: 1;
    if ($this->species === 'dog') {
        // placedog.net – стабильный сервис фото собак, детерминированно по id
        return 'https://placedog.net/640/480?id=' . (($id % 200) + 1);
    }
    $keyword = $this->species === 'other' ? 'parrot,bird' : 'cat,kitten';
    // loremflickr – настоящие фото по ключевому слову, lock делает выбор
    постоянным
    return 'https://loremflickr.com/640/480/' . $keyword . '?lock=' . $id;
}

/** Эмодзи-заглушка на случай, если фото не загрузилось. */
public function getPhotoEmoji(): string
{
    $val = trim((string) $this->photoUrl);
    if ($val !== '' && strpos($val, 'http') !== 0 && strpos($val, '/') !== 0) {
        return $val; // в photo_url лежит эмодзи
    }
    return ['cat' => '🐱', 'dog' => '🐶', 'other' => '🐾'][$this->species] ?? '🐾';
}

// ----- Человекочитаемые ярлыки -----
public function getSpeciesLabel(): string
{
    return ['cat' => 'Кошка', 'dog' => 'Собака', 'other' => 'Другое'][$this->species] ?? $this->species;
}

public function getGenderLabel(): string
{
    return ['male' => 'Мальчик', 'female' => 'Девочка'][$this->gender] ?? $this->gender;
}

public function getActivityLabel(): string
{
    return ['low' => 'Низкая', 'normal' => 'Средняя', 'high' => 'Высокая'][$this->activity] ?? $this->activity;
}

public function getStatusLabel(): string
{
    return ['available' => 'Ищет дом', 'reserved' => 'Забронирован', 'adopted' => 'Дома'][$this->status] ?? $this->status;
}

/** Возраст словами: «2 года 3 мес.» */
public function getAgeLabel(): string
{
    $years = intval($this->ageMonths, 12);
    $months = $this->ageMonths % 12;
    $parts = [];
    if ($years > 0) {

```

```

        $parts[] = $years . ' ' . self::plural($years, 'год', 'года', 'лет');
    }
    if ($months > 0) {
        $parts[] = $months . ' ' . self::plural($months, 'месяц', 'месяца',
'mесяцев');
    }
    return $parts ? implode(' ', $parts) : 'меньше месяца';
}

private static function plural(int $n, string $one, string $few, string $many):
string
{
    $n = abs($n) % 100;
    $n1 = $n % 10;
    if ($n > 10 && $n < 20) return $many;
    if ($n1 > 1 && $n1 < 5) return $few;
    if ($n1 === 1) return $one;
    return $many;
}

protected static function getTableName(): string
{
    return 'pets';
}
}

```

Листинг Б.6. Модель «Питомец» — пример сущности

src/MyProject/View/View.php

```

<?php

namespace MyProject\View;

class View
{
    /** @var string */
    private $templatesPath;

    public function __construct(string $templatesPath)
    {
        $this->templatesPath = $templatesPath;
    }

    /**
     * Рендерит шаблон с переданными переменными.
     *
     * @param string $templateName Имя файла шаблона
     * @param array $vars Переменные, доступные внутри шаблона
     * @param int $code HTTP-код ответа (по умолчанию 200)
     */
    public function renderHtml(string $templateName, array $vars = [], int $code =
200): void
    {
        http_response_code($code);
        extract($vars);

        ob_start();
        include $this->templatesPath . '/' . $templateName;
        $buffer = ob_get_contents();
        ob_end_clean();

        echo $buffer;
    }
}

```

Листинг Б.7. Компонент представления (рендеринг шаблонов)

src/MyProject/Controllers/MainController.php

```
<?php

namespace MyProject\Controllers;

use MyProject\Models\Pets\Pet;
use MyProject\Models\Articles\Article;

class MainController extends AbstractController
{
    /** Главная страница: герой-блок, свежие питомцы, последние статьи. */
    public function main(): void
    {
        $pets = Pet::findAll();
        // показываем до 6 питомцев, ищущих дом
        $available = array_filter($pets, function (Pet $p) {
            return $p->getStatus() === 'available';
        });
        $featuredPets = array_slice(array_values($available), 0, 6);

        $articles = Article::findAll();
        $latestArticles = array_slice($articles, 0, 3);

        $stats = [
            'total' => count($pets),
            'available' => count($available),
            'adopted' => count(array_filter($pets, function (Pet $p) {
                return $p->getStatus() === 'adopted';
            })),
        ];

        $this->render('main/main.php', [
            'title' => 'Лапки – приют для животных',
            'featuredPets' => $featuredPets,
            'latestArticles' => $latestArticles,
            'stats' => $stats,
        ]);
    }

    // ДЗ3 (роутинг): экшн sayHello – «Привет, $name»
    public function sayHello(string $name): void
    {
        // ДЗ4: для этой страницы задаём собственный title
        $this->render('main/hello.php', [
            'name' => $name,
            'title' => 'Страница приветствия',
        ]);
    }

    // ДЗ3 (роутинг): новый экшн sayBye – «Пока, $name»
    public function sayBye(string $name): void
    {
        echo 'Пока, ' . htmlspecialchars($name);
    }

    /**
     * Лаба 1: демонстрация функции get_headers().
     * Выводит HTTP-заголовки ответа для указанного URL.
     */
    public function headers(): void
    {
        $defaultUrl = 'https://mospolytech.ru';
        $url = isset($_GET['url']) && trim($_GET['url']) !== '' ? trim($_GET['url']) :
        $defaultUrl;

        $headers = @get_headers($url, 1);

        if ($headers === false) {
            $output = "Не удалось получить заголовки с адреса: {$url}\n";
        }
    }
}
```

```

        . "Проверьте корректность URL и доступность ресурса.";
    } else {
        $output = "URL: {$url}\n" . str_repeat('-', 50) . "\n\n";
        foreach ($headers as $key => $value) {
            if (is_array($value)) {
                $value = implode(' ', $value);
            }
            $output .= is_int($key) ? $value . "\n" : $key . ': ' . $value . "\n";
        }

        $this->render('main/headers.php', [
            'title' => 'HTTP-заголовки (get_headers)',
            'url'    => $url,
            'output' => $output,
        ]);
    }
}

```

Листинг Б.8. Главный контроллер: главная страница, sayHello, sayBye, страница get_headers

src/MyProject/Controllers/ArticlesController.php

```

<?php

namespace MyProject\Controllers;

use MyProject\Models\Articles\Article;
use MyProject\Models\Users\User;
use MyProject\Exceptions\NotFoundException;

class ArticlesController extends AbstractController
{
    /** Список статей-гайдов. */
    public function list(): void
    {
        $this->render('articles/list.php', [
            'title' => 'Статьи и советы — Лапки',
            'articles' => Article::findAll(),
        ]);
    }

    /**
     * Детальная страница статьи.
     * ДЗ5: после получения статьи делаем дополнительный запрос
     * за автором из таблицы users и выводим его nickname.
     */
    public function show(int $articleId): void
    {
        $article = Article::getById($articleId);
        if ($article === null) {
            throw new NotFoundException('Статья не найдена');
        }

        // ДЗ5: получаем автора статьи
        $author = User::getById($article->getAuthorId());

        // ДЗ4: title статьи задаётся динамически
        $this->render('articles/show.php', [
            'title' => $article->getName(),
            'article' => $article,
            'author' => $author,
        ]);
    }
}

```

Листинг Б.9. Контроллер статей (получение автора из users — ДЗ 5)

src/MyProject/Controllers/CalculatorController.php

```
<?php

namespace MyProject\Controllers;

class CalculatorController extends AbstractController
{
    /**
     * Калькулятор суточной нормы корма.
     * Расчёт данных на основе параметров (вес, вид, возраст, активность).
     * Параметры приходят GET-запросом, результат считается на сервере (PHP).
     */
    public function feed(): void
    {
        $result = null;
        $params = [
            'weight' => $_GET['weight'] ?? '',
            'species' => $_GET['species'] ?? 'cat',
            'age' => $_GET['age'] ?? 'adult',
            'activity' => $_GET['activity'] ?? 'normal',
        ];

        // Если переданы корректные параметры – выполняем расчёт
        if (isset($_GET['weight']) && is_numeric($_GET['weight']) && (float)
            $_GET['weight'] > 0) {
            $weight = (float) $_GET['weight'];
            $species = in_array($params['species'], ['cat', 'dog'], true) ?
                $params['species'] : 'cat';
            $age = in_array($params['age'], ['baby', 'adult', 'senior'], true) ?
                $params['age'] : 'adult';
            $activity = in_array($params['activity'], ['low', 'normal', 'high'], true)
                ? $params['activity'] : 'normal';

            // Базовая норма: г сухого корма на кг веса
            $basePerKg = $species === 'cat' ? 20 : 25;

            // Коэффициент возраста
            $ageFactor = ['baby' => 1.6, 'adult' => 1.0, 'senior' => 0.85][$age];
            // Коэффициент активности
            $activityFactor = ['low' => 0.9, 'normal' => 1.0, 'high' =>
                1.2][$activity];

            $gramsPerDay = $weight * $basePerKg * $ageFactor * $activityFactor;
            $gramsPerDay = (int) round($gramsPerDay);

            // Примерные калории: ~3.5 ккал на грамм сухого корма
            $kcalPerDay = (int) round($gramsPerDay * 3.5);

            // Рекомендуемое число кормлений
            $meals = $age === 'baby' ? 4 : ($age === 'senior' ? 2 : 3);

            $result = [
                'grams' => $gramsPerDay,
                'kcal' => $kcalPerDay,
                'meals' => $meals,
                'gramsPerMeal' => (int) round($gramsPerDay / $meals),
            ];
        }

        $this->render('calculator/feed.php', [
            'title' => 'Калькулятор корма – Лапки',
            'params' => $params,
            'result' => $result,
        ]);
    }
}
```

Листинг Б.10. Контроллер калькулятора нормы корма